

AI Assisted Coding Ass-18.3

B.Sushanth

240A52L11-B50

Task1:

Code and prompt:

```
#Task 1
"""Prompt: Write a Python function that retrieves fare details for a given source and destination using a public transportation API.
|   |   |   The function should handle potential errors such as network issues, invalid responses, and timeouts gracefully."""

import requests
def get_fare_details(source, destination):
    api_url = f"https://api.publictransport.com/fare?source={source}&destination={destination}"

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors

        fare_data = response.json()
        if 'fare' in fare_data:
            print(f"Fare from {source} to {destination}: {fare_data['fare']}")
        else:
            print("Fare information not available.")

    except requests.exceptions.Timeout:
        print("The request timed out. Please try again later.")
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.RequestException as err:
        print(f"An error occurred: {err}")
    except ValueError:
        print("Invalid response received from the API.")

# Example usage
source_station = input("Enter source station: ")
destination_station = input("Enter destination station: ")
get_fare_details(source_station, destination_station)
```

Task2:

Code and prompt:

```
"""Prompt: Write a Python function that converts a given amount in Indian Rupees (INR) to US Dollars (USD), Euros (EUR), and British Pounds (GBP) using a currency conversion API.
|   |   |   The function should handle potential errors such as network issues, invalid responses, and timeouts gracefully.
|   |   |   The results should be displayed in a tabular format using the pandas library."""

import requests
import pandas as pd
def convert_currency(amount_inr):
    api_url = "https://api.exchangerate-api.com/v4/latest/INR"

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors

        exchange_rates = response.json().get('rates', {})
        if not exchange_rates:
            print("Exchange rates not available.")
            return

        conversion_results = {
            'Currency': ['USD', 'EUR', 'GBP'],
            'Amount': [amount_inr * exchange_rates.get('USD', 0),
                        amount_inr * exchange_rates.get('EUR', 0),
                        amount_inr * exchange_rates.get('GBP', 0)]
        }

        df = pd.DataFrame(conversion_results)
        print(df)

    except requests.exceptions.Timeout:
        print("The request timed out. Please try again later.")
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.RequestException as err:
        print(f"An error occurred: {err}")
    except ValueError:
        print("Invalid response received from the API.")

# Example usage
try:
    amount_inr = float(input("Enter amount in INR: "))
    convert_currency(amount_inr)
except ValueError:
    print("Invalid input. Please enter a numeric value for the amount.")
```

Task3:

Code and prompt:

```
#Task 3
"""Prompt: Write a Python function that retrieves information about a GitHub repository, including its name, description, number of stars, forks, and open issues using the GitHub API.
| | | The function should handle potential errors such as network issues, invalid responses, and timeouts gracefully."""
import requests

def fetch_github_repo_info(owner, repo):
    api_url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors

        repo_data = response.json()
        print(f"Repository Name: {repo_data.get('name', 'N/A')}")
        print(f"Description: {repo_data.get('description', 'N/A')}")
        print(f"Stars: {repo_data.get('stargazers_count', 'N/A')}")
        print(f"Forks: {repo_data.get('forks_count', 'N/A')}")
        print(f"Open Issues: {repo_data.get('open_issues_count', 'N/A')}")

    except requests.exceptions.HTTPError as http_err:
        if response.status_code == 404:
            print("Repository not found. Please check the owner and repository name.")
        elif response.status_code == 403 and 'X-RateLimit-Remaining' in response.headers and response.headers['X-RateLimit-Remaining'] == '0':
            print("Rate limit exceeded. Please try again later.")
        else:
            print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.Timeout:
        print("The request timed out. Please try again later.")
    except requests.exceptions.RequestException as err:
        print(f"An error occurred: {err}")
    except ValueError:
        print("Invalid response received from the API.")

# Example usage
owner = input("Enter GitHub repository owner: ")
repo = input("Enter GitHub repository name: ")
fetch_github_repo_info(owner, repo)
```

Task4:

Code and prompt:

```
#Task 4
"""Prompt: Write a Python function that retrieves the top 5 news headlines for a given category (e.g., sports, technology, health) using a news API.
| | | The function should handle potential errors such as network issues, invalid responses, and timeouts gracefully."""
import requests

def fetch_news_headlines(category) :
    api_key = "your_api_key_here"
    api_url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={api_key}"

    try :
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors
        news_data = response.json()
        if news_data.get('status') != 'ok' :
            print("Error fetching news: ", news_data.get('message', 'Unknown error'))
            return
        articles = news_data.get('articles', [])[:5]
        if not articles :
            print("No headlines found for the given category.")
            return
        print(f"Top 5 headlines in {category} category:")
        for idx, article in enumerate(articles, start=1) :
            print(f"{idx}. {article.get('title', 'No title available')}")
    except requests.exceptions.HTTPError as http_err :
        if response.status_code == 401 :
            print("Invalid API key. Please check your API key and try again.")
        elif response.status_code == 400 :
            print("Invalid category. Please choose from sports, technology, or health.")
        else :
            print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.Timeout :
        print("The request timed out. Retrying...")
        fetch_news_headlines(category) # Retry the request
    except requests.exceptions.RequestException as err :
        print(f"An error occurred: {err}")

# Example usage
category = input("Enter news category (sports, technology, health): ")
fetch_news_headlines(category)
```

Task5:

Code and prompt:

```
#Task 5
"""Prompt: Write a Python function that retrieves COVID-19 statistics for a given country using a public API.
|   |   |   The function should handle potential errors such as network issues, invalid responses, and timeouts gracefully."""

import requests
def fetch_covid_stats(country):
    api_url = f"https://disease.sh/v3/covid-19/countries/{country}"

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors

        covid_data = response.json()
        print(f"COVID-19 Statistics for {country.capitalize()}:")
        print(f"Total Confirmed Cases: {covid_data.get('cases', 'N/A')}")
        print(f"Total Deaths: {covid_data.get('deaths', 'N/A')}")
        print(f"Total Recovered Cases: {covid_data.get('recovered', 'N/A')}")
        print(f"Active Cases: {covid_data.get('active', 'N/A')}")

    except requests.exceptions.HTTPError as http_err:
        if response.status_code == 404:
            print("Invalid country name. Please check the country name and try again.")
        elif response.status_code == 429:
            print("API rate limit exceeded. Please wait and try again later.")
        else:
            print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.Timeout:
        print("The request timed out. Retrying...")
        fetch_covid_stats(country) # Retry the request
    except requests.exceptions.RequestException as err:
        print(f"An error occurred: {err}")

# Example usage
country_name = input("Enter country name to fetch COVID-19 statistics: ")
fetch_covid_stats(country_name)
```